Assignment 1 Report

Zhao TONG

1.Problem Understanding

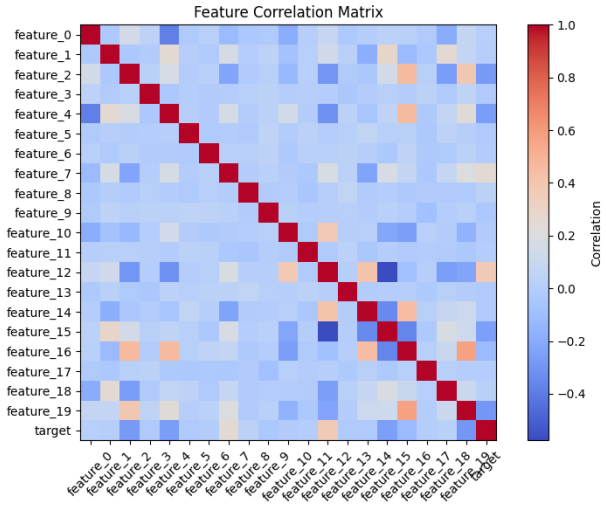
This project aims to build a complete machine learning classification pipeline to better understand how each component of a learning system works. The model is logistic regression, a fundamental and powerful approach for binary classification tasks. The dataset consists of 1,700 training samples and 300 test samples, each containing 20 numerical features and one target variable (0 or 1). The primary objective is to predict the target class as accurate as possible and to explore techniques that can enhance model performance.

2.Exploratory Data Analysis

Before training, an exploratory data analysis was conducted to understand the dataset's structure, quality, and feature relationships. The analysis included summary statistics, missing-value checks, and visualizations.

Dataset structure: The training data contains 1,700 samples and 21 columns (20 features + 1 target). All features are numerical and continuous. No missing values were found. This indicates that the data is suitable for direct modeling.

Feature statistics: The descriptive summary shows that most features are roughly centered around zero, with standard deviations close to 2.0. Several features exhibit wide ranges (like from -6 to +8), which reinforces the importance of standardization before model training.



Target distribution: As shown in left figure, the target classes are nearly balanced. This ensures that accuracy is a reliable performance metric without the need for class weighting.

Feature correlations: The correlation heatmap reveals that most features are weakly correlated, indicating little redundancy. This supports the assumption of approximate feature independence, making logistic regression a reasonable model.

3.Model Building and Training & Results and Analysis

First, I accomplished all the TODO tasks as required, and the principals behind each function are commented in the code file. The initial accuracy and AUC are 0.7633 and 0.8484 respectively with the original parameters in User Config. These results served as the baseline for subsequent optimization.

```
>> Test Accuracy: 0.7633
>> Confusion Matrix [[TN, FP], [FN, TP]]:
[[128  36]
 [ 35 101]]
>> Test AUC: 0.8484
```

Then, I implemented hyperparameter tuning by experimenting with different learning rates, thresholds, and numbers of epochs with the help of function tune_on_validate. I adjusted epochs ranging from 8000 to 14000 manually, then call this function to perform a grid search over learning rates in LR_CAND=[0.0001, 0.0003, 0.0005, 0.0007, 0.001, 0.003, 0.005], thresholds in 0.1-0.9. The best result I obtained from this tuning process was accuracy=0.7867 and AUC=0.8545, corresponding to epochs=11100, LR=0.001, and threshold=0.48. Epochs fewer than 10000 did not reach the minimum loss, while those exceeding 12000 tended to overfit. The configuration with 11000 epochs achieved a nearly identical best result, with AUC = 0.8544.

I further experimented with L2 regularization learned in Section 1 by adding a penalty term $\lambda\|W\|^2$ to the loss function, and augmented my tune_on_validate function so it can search all combinations of LR_CAND and L2_CAND=[0.0, 0.0001, 0.001, 0.01, 0.1, 1, 10, 100]. However, small λ shown no impact to accuracy and AUC, and both accuracy and AUC dropped as λ increased when $\lambda \geq 10$, suggesting that the model was already stable and did not require extra regularization.

```
>> Test Accuracy: 0.7867
>> Confusion Matrix [[TN, FP], [FN, TP]]:
[[125  39]
 [ 25 111]]
>> Test AUC: 0.8545
```

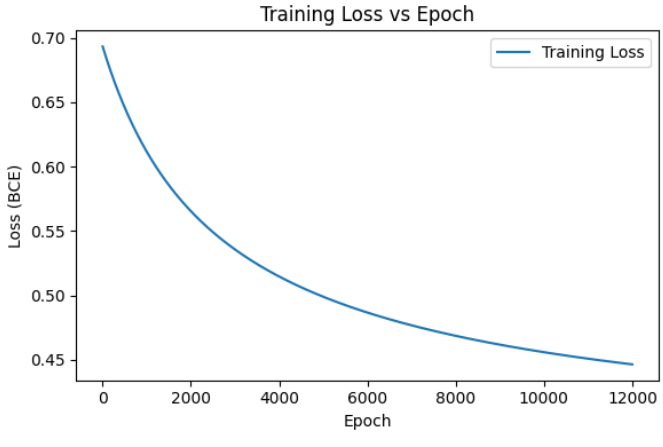
Next, I moved on to try the k-fold cross-validation (introduced in Section 2) in order to obtain a more reliable estimate of the model's generalization ability and to reduce the validation–test inconsistency I observed previously (Large LR perform better on val but worse on test than smaller LR). And to maximize data utilization, I merged the original training and validation sets before performing k-fold, ensuring that each sample was used for both training and validation in different folds. I tested k=3, 5, 7, and it turned out that k=5 performs the best. This process achieved a mean validation accuracy 0.7565, while the final model retrained on the full training set reached Test Accuracy = 0.7867 and Test AUC = 0.8526 (with epochs=11000).

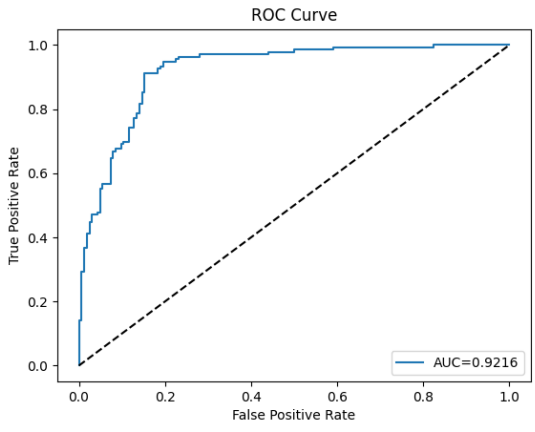
```
>> [k-Fold Tuning] Best mean Val Acc=0.7565 with lr=0.0006, l2=0.0, thr=0.472
>> Test Accuracy: 0.7867
>> Confusion Matrix [[TN, FP], [FN, TP]]:
[[124  40]
 [ 24 112]]
>> Test AUC: 0.8526
```

Although the final test performance did not surpass that of the single validation split, k-fold validation made the evaluation more reliable and consistent by reducing the effect of any single data split. This shows that the performance limit was not due to overfitting or random validation results, but rather because a linear logistic regression model cannot capture more complex relationships in the data.

To address this limitation, I expanded the feature set by adding squared terms to introduce simple nonlinearity while keeping the model interpretable and convex. After re-tuning with 5-fold cross-validation, the model achieved a mean validation accuracy of 0.8188, and when retrained on the full training set, reached Test Accuracy = 0.8733 and AUC = 0.9216 (epochs=12000). This result shows that adding nonlinear features significantly improved model performance.

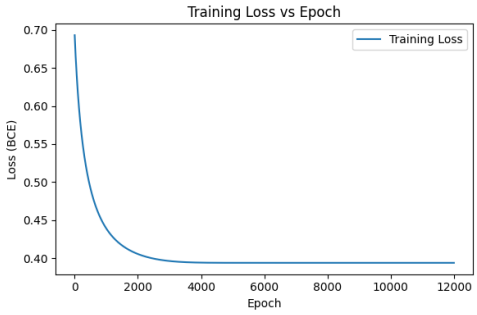
```
>> [k-Fold Tuning] Best mean Val Acc=0.8188 with lr=0.0005, l2=0.0, thr=0.460
>> Test Accuracy: 0.8733
>> Confusion Matrix [[TN, FP], [FN, TP]]:
[[138  26]
 [ 12 124]]
>> Test AUC: 0.9216
```





Last, I wanted to see if the Adam optimizer introduced in Section 3 would further improve the model's convergence and performance. Adam successfully reduced the final training loss from around 0.45 to 0.40, showing faster convergence than vanilla gradient descent. However, the test accuracy dropped to 0.8367 and AUC to 0.9195, indicating that while Adam optimized the training objective more aggressively, it did not generalize better to unseen data. This suggests that for relatively small and clean datasets, a simple gradient descent optimizer may provide a better balance between convergence stability and generalization.

```
>> Test Accuracy: 0.8367
>> Confusion Matrix [[TN, FP], [FN, TP]]:
[[136  28]
 [ 21 115]]
>> Test AUC: 0.9195
```



4.Challenges and Solutions

The main challenge of this project was model tuning. Early experiments showed that larger learning rates improved validation accuracy but reduced test accuracy. To investigate this, I introduced L2 regularization and later k-fold cross-validation, which confirmed that the model was already stable and not overfitting, though k-fold mainly improved evaluation reliability rather than absolute accuracy.

In addition, the model's performance was limited by its linear nature, so I added squared features which effectively addressed this limitation and became the key step that pushed accuracy above 0.87.

5.Conclusion and Future work

This project successfully built a complete machine learning pipeline from scratch, including data exploration, model derivation, training, and evaluation. Starting from a baseline logistic regression, I improved the model through hyperparameter tuning, validation, and feature engineering, achieving a final Test Accuracy of 0.8733 and AUC of 0.9216.

Future work may include regularization with feature selection, or extending the model to multi-layer to capture more complex relationships.